
**РАБОЧИЙ ПРОТОКОЛ И ОТЧЁТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2 "[C++
и UNIX]: C++ BUILD / IF / LOOP, PYTHON"**

Группа: Z3344
Студенты: Поздеев Дмитрий
Преподаватель: Маслов И.

К работе допущен:
Работа выполнена:
Отчет принят:

1 Цель работы

- Познакомить студента с принципами компиляции исходного кода.
- Составить программу с использованием циклов, условий и функций.
- Сравнить быстродействие между C++ и Python. Ознакомление с типами данных.

2 Задачи, решаемые при выполнении работы

- C++ EXPRESSION Создать и скомпилировать программу на C++
- PYTHON EXPRESSION Создать и скомпилировать программу на Python 3
- SAVE Результат всех вышеперечисленных шагов сохранить в репозиторий (+ отчет по данной ЛР в папку doc)

3 Блок на C++

3.1 Директории и файлы доступа

Листинг 1: Код на плюсах

```
#include <iostream>  
#include <chrono>  
#include <limits>
```

```
#include <string>
```

```
double calculate(double x) {  
    return x*x - x*x + x*4 - x*5 + x + x;  
}
```

```
int main() {  
    const double x = 3.0;
```

```
    while(true) {  
        std::cout << "Enter_iterations_count_(or_'q'_to_quit):_";  
        std::string input;  
        std::cin >> input;
```

```
        // 1.  
        if(input == "q" || input == "Q") {  
            std::cout << "Exiting_program..." << std::endl;  
            break;  
        }
```

```
        // 2. —  
        bool is_valid = true;  
        for(char c : input) {  
            if(!isdigit(c)) {  
                is_valid = false;  
                break;  
            }  
        }
```

```
        if(!is_valid) {  
            std::cerr << "Error:_Input_must_be_positive_integer!\n\n";  
            std::cin.clear();  
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');  
            continue;  
        }
```

```
        // 3.  
        try {  
            int n = std::stoi(input);  
  
            if(n <= 0) {  
                std::cerr << "Error:_Number_must_be_greater_than_zero!\n\n";  
                continue;  
            }  
        }
```

```
        // 4.  
        auto start = std::chrono::high_resolution_clock::now();  
        for(int i = 0; i < n; ++i) {  
            volatile double result = calculate(x);  
        }
```

```

        auto end = std::chrono::high_resolution_clock::now();

        // 5.
        auto duration = std::chrono::duration_cast<std::chrono::millisecond>
            (std::chrono::duration_cast<std::chrono::microsecond>(end - start) / 1000);
        std::cout << "Time:_" << duration.count() << "_ms\n" << std::endl;

        } catch(const std::exception& e) {
            std::cerr << "Error:_" << e.what() << "\n\n";
        }
    }

    return 0;
}

```

3.2 Блок на питоне

Листинг 2: Код на Python

```

import time

def calculate(x):
    return x**2 - x**2 + 4*x - 5*x + x + x
x = 3.0
while True:
    user_input = input("Enter iterations count_(or 'q' to quit):_").strip()
    # 1.
    if user_input.lower() == 'q':
        print("Exiting program...")
        break
    # 2.
    try:
        n = int(user_input)
    except ValueError:
        print("Error:_Input_must_be_an_integer!\n")
        continue
    # 3.
    if n <= 0:
        print("Error:_Number_must_be_greater_than_zero!\n")
        continue
    # 4.
    start = time.perf_counter()
    result = calculate(x) #
    for _ in range(n): pass #
    end = time.perf_counter()
    # 5.
    print(f"Result:_{result}")
    print(f"Time:_{end-start:.6f}_seconds\n")

```

3.3 Работа с Git

Листинг 3: Клонирование репозитория и создание структуры

```
git clone https://github.com/PozdeevDA/my_labs.git
cd my_labs
mkdir -p lab_01/{build,src,doc,cmake} lab_02/{build,src,doc,cmake}
git checkout -b dev
git push -u origin dev
git branch -d main
git push origin --delete main
```

Листинг 4: Скрипт для переноса из dev в stg:

```
git add .
git commit -m "last_fix"
git push origin dev
```

4 Выводы

В процессе работы я на собственном опыте убедился, что C++ обеспечивает высокую производительность за счет компиляции в нативный код, тогда как Python предлагает удобство и скорость разработки. Это подтверждает, что выбор инструмента зависит от требований задачи, а использование Git и четкой структуры проекта критически важно для эффективной работы.